

CSS Table Properties: Complete Guide

CSS Learning Resources

May 9, 2026

Contents

1	Introduction	2
1.1	Why Table Properties Matter	2
2	Table HTML Structure	2
2.1	Semantic Table Markup	2
3	Core Table Properties	3
3.1	Border-Collapse	3
3.2	Border-Spacing	3
3.3	Table-Layout	4
4	Real-World Examples	4
4.1	Example 1: Clean Data Table	4
4.2	Example 2: Striped Rows	5
4.3	Example 3: Fixed Header	6
4.4	Example 4: Responsive Table	6
4.5	Example 5: Financial Table	7
4.6	Example 6: Sortable Table Headers	8
5	Accessibility for Tables	9
5.1	Header Associations	9
5.2	Scope Attribute	9
5.3	Caption and Summary	9
6	Browser Support and Performance	10
6.1	Compatibility Table	10
6.2	Performance Tips	10
7	Common Mistakes	10
7.1	Mistake 1: Missing Table Structure	10
7.2	Mistake 2: Table for Layout	11
7.3	Mistake 3: Not Making Tables Responsive	11
7.4	Mistake 4: Insufficient Color Contrast	11
7.5	Mistake 5: Ignoring Accessibility	12

8	Quick Reference	12
8.1	Table Styling Patterns	12
8.2	Production Checklist	13
9	Summary	13

1 Introduction

Tables are fundamental HTML elements for displaying structured data. CSS table properties provide comprehensive control over how tables render, enabling developers to create accessible, visually appealing data presentations. While tables were historically misused for layout (a practice now discouraged), modern CSS table properties focus on styling data tables appropriately for their semantic purpose.

Understanding CSS table properties is essential for presenting complex data in clear, organized formats. Combined with modern responsive techniques, tables can adapt gracefully to different screen sizes while maintaining readability and accessibility.

1.1 Why Table Properties Matter

- **Data Presentation:** Display structured data clearly and organized
- **Accessibility:** Semantic tables support screen readers and assistive tech
- **Readability:** Proper styling improves comprehension of complex data
- **Responsiveness:** Tables adapt to mobile and desktop screens
- **User Experience:** Well-styled tables guide users through information
- **Print Friendly:** Tables render appropriately in print layouts
- **Performance:** Proper CSS reduces rendering complexity

2 Table HTML Structure

2.1 Semantic Table Markup

```
1 <table>
2   <thead>
3     <tr>
4       <th>Header 1</th>
5       <th>Header 2</th>
6       <th>Header 3</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
11      <td>Data 1</td>
12      <td>Data 2</td>
13      <td>Data 3</td>
14    </tr>
15    <tr>
16      <td>Data 4</td>
17      <td>Data 5</td>
18      <td>Data 6</td>
19    </tr>
20  </tbody>
21  <tfoot>
22    <tr>
```

```
23         <td>Footer 1</td>
24         <td>Footer 2</td>
25         <td>Footer 3</td>
26     </tr>
27 </tfoot>
28 </table>
```

3 Core Table Properties

3.1 Border-Collapse

Controls how table borders are rendered:

```
1  /* Separate borders (default) */
2  table {
3      border-collapse: separate;
4      border-spacing: 0;
5  }
6
7  /* Collapsed borders */
8  table {
9      border-collapse: collapse;
10 }
11
12 /* Example: Collapse borders */
13 table {
14     border-collapse: collapse;
15     width: 100%;
16 }
17
18 th, td {
19     border: 1px solid #ddd;
20     padding: 0.75rem;
21 }
```

3.2 Border-Spacing

Defines space between cells in separate border model:

```
1  /* Default spacing */
2  table {
3      border-collapse: separate;
4      border-spacing: 0;
5  }
6
7  /* Horizontal and vertical spacing */
8  table {
9      border-collapse: separate;
10     border-spacing: 10px 5px;
11 }
12
13 /* Equal spacing all directions */
14 table {
```

```
15     border-collapse: separate;
16     border-spacing: 8px;
17 }
```

3.3 Table-Layout

Controls how browsers calculate table column widths:

```
1  /* Auto layout (default) - considers content) */
2  table {
3      table-layout: auto;
4  }
5
6  /* Fixed layout - uses column widths */
7  table {
8      table-layout: fixed;
9      width: 100%;
10 }
11
12 .col-narrow {
13     width: 15%;
14 }
15
16 .col-wide {
17     width: 85%;
18 }
```

4 Real-World Examples

4.1 Example 1: Clean Data Table

```
1  <table class="data-table">
2      <thead>
3          <tr>
4              <th>Name</th>
5              <th>Email</th>
6              <th>Status</th>
7              <th>Date</th>
8          </tr>
9      </thead>
10     <tbody>
11         <tr>
12             <td>John Doe</td>
13             <td>john@example.com</td>
14             <td><span class="badge active">Active</span></td>
15             <td>2024-01-15</td>
16         </tr>
17     </tbody>
18 </table>
```

```
1 .data-table {
```

```
2     width: 100%;
3     border-collapse: collapse;
4     font-size: 0.95rem;
5 }
6
7 .data-table thead {
8     background: #f5f5f5;
9     border-bottom: 2px solid #ddd;
10 }
11
12 .data-table th {
13     padding: 1rem;
14     text-align: left;
15     font-weight: 600;
16     color: #333;
17 }
18
19 .data-table td {
20     padding: 0.75rem 1rem;
21     border-bottom: 1px solid #eee;
22 }
23
24 .data-table tbody tr:hover {
25     background: #f9f9f9;
26 }
27
28 .badge {
29     display: inline-block;
30     padding: 0.25rem 0.75rem;
31     border-radius: 12px;
32     font-size: 0.85rem;
33     font-weight: 600;
34 }
35
36 .badge.active {
37     background: #d4edda;
38     color: #155724;
39 }
```

4.2 Example 2: Striped Rows

```
1  /* Alternate row colors */
2  table tbody tr:nth-child(odd) {
3      background: white;
4  }
5
6  table tbody tr:nth-child(even) {
7      background: #f9f9f9;
8  }
9
10 /* Hover effect */
11 table tbody tr:hover {
12     background: #f0f0f0;
```

```
13 }
```

4.3 Example 3: Fixed Header

```
1 .table-container {
2     max-height: 400px;
3     overflow-y: auto;
4 }
5
6 table {
7     width: 100%;
8     border-collapse: collapse;
9 }
10
11 thead {
12     position: sticky;
13     top: 0;
14     background: white;
15     z-index: 10;
16     box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
17 }
18
19 th {
20     padding: 1rem;
21     text-align: left;
22     font-weight: 600;
23 }
24
25 td {
26     padding: 0.75rem 1rem;
27     border-bottom: 1px solid #eee;
28 }
```

4.4 Example 4: Responsive Table

```
1 /* Desktop: Normal table */
2 table {
3     width: 100%;
4     border-collapse: collapse;
5 }
6
7 th, td {
8     padding: 1rem;
9     text-align: left;
10    border-bottom: 1px solid #ddd;
11 }
12
13 /* Mobile: Card layout */
14 @media (max-width: 768px) {
15     table, thead, tbody, th, td, tr {
16         display: block;
```

```
17     }
18
19     thead tr {
20         position: absolute;
21         top: -9999px;
22         left: -9999px;
23     }
24
25     tr {
26         margin-bottom: 1rem;
27         border: 1px solid #ddd;
28         border-radius: 4px;
29     }
30
31     td {
32         padding-left: 50%;
33         position: relative;
34         border: none;
35     }
36
37     td::before {
38         content: attr(data-label);
39         position: absolute;
40         left: 1rem;
41         font-weight: 600;
42         color: #666;
43     }
44 }
```

4.5 Example 5: Financial Table

```
1 .financial-table {
2     width: 100%;
3     border-collapse: collapse;
4     font-variant-numeric: tabular-nums;
5 }
6
7 .financial-table th {
8     background: #2c3e50;
9     color: white;
10    padding: 1rem;
11    text-align: right;
12 }
13
14 .financial-table th:first-child {
15     text-align: left;
16 }
17
18 .financial-table td {
19     padding: 0.75rem 1rem;
20     text-align: right;
21     border-bottom: 1px solid #eee;
22 }
```



```
23
24 .financial-table td:first-child {
25     text-align: left;
26     font-weight: 500;
27 }
28
29 .positive {
30     color: #28a745;
31 }
32
33 .negative {
34     color: #dc3545;
35 }
36
37 .financial-table tfoot {
38     background: #f5f5f5;
39     font-weight: 600;
40 }
```

4.6 Example 6: Sortable Table Headers

```
1  /* Sortable header styling */
2  .sortable th {
3      cursor: pointer;
4      user-select: none;
5      padding-right: 1.5rem;
6      position: relative;
7  }
8
9  .sortable th:hover {
10     background: #e8e8e8;
11 }
12
13 /* Sort indicator icons */
14 .sortable th::after {
15     content: '';
16     position: absolute;
17     right: 0.5rem;
18     top: 50%;
19     transform: translateY(-50%);
20     opacity: 0;
21 }
22
23 .sortable th.asc::after {
24     content: '';
25     opacity: 1;
26 }
27
28 .sortable th.desc::after {
29     content: '';
30     opacity: 1;
31 }
```

5 Accessibility for Tables

5.1 Header Associations

```
1 <!-- Associate headers with data -->
2 <table>
3   <thead>
4     <tr>
5       <th id="name">Name</th>
6       <th id="email">Email</th>
7       <th id="status">Status</th>
8     </tr>
9   </thead>
10  <tbody>
11    <tr>
12      <td headers="name">John Doe</td>
13      <td headers="email">john@example.com</td>
14      <td headers="status">Active</td>
15    </tr>
16  </tbody>
17 </table>
```

5.2 Scope Attribute

```
1 <table>
2   <thead>
3     <tr>
4       <th scope="col">Column 1</th>
5       <th scope="col">Column 2</th>
6     </tr>
7   </thead>
8   <tbody>
9     <tr>
10      <th scope="row">Row 1</th>
11      <td>Data</td>
12    </tr>
13  </tbody>
14 </table>
```

5.3 Caption and Summary

```
1 <table>
2   <caption>Sales Data for Q1 2024</caption>
3   <thead>
4     <tr>
5       <th>Month</th>
6       <th>Revenue</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
```

```

11         <td>January</td>
12         <td>$50,000</td>
13     </tr>
14 </tbody>
15 </table>

```

6 Browser Support and Performance

6.1 Compatibility Table

Feature	Chrome	Firefox	Safari	Edge
border-collapse	All	All	All	All
border-spacing	All	All	All	All
table-layout	All	All	All	All

6.2 Performance Tips

```

1  /* Use fixed layout for known widths */
2  table {
3      table-layout: fixed;
4      width: 100%;
5  }
6
7  /* Avoid complex selectors */
8  /* Bad: table tbody tr td:nth-child(3) */
9  /* Good: .priority-column */
10
11 /* Minimize reflows */
12 table {
13     border-collapse: collapse;
14 }
15
16 /* Use CSS containment */
17 table {
18     contain: layout style paint;
19 }

```

7 Common Mistakes

7.1 Mistake 1: Missing Table Structure

Wrong:

```

1 <table>
2     <tr><th>Header</th></tr>
3     <tr><td>Data</td></tr>
4 </table>

```

Correct:

```
1 <table>
2   <thead><tr><th>Header</th></tr></thead>
3   <tbody><tr><td>Data</td></tr></tbody>
4 </table>
```

7.2 Mistake 2: Table for Layout

Bad practice (avoid):

```
1 <table>
2   <tr>
3     <td>Navigation</td>
4     <td>Content</td>
5     <td>Sidebar</td>
6   </tr>
7 </table>
```

Use CSS Grid instead:

```
1 .layout {
2   display: grid;
3   grid-template-columns: 200px 1fr 250px;
4   gap: 1rem;
5 }
```

7.3 Mistake 3: Not Making Tables Responsive

Problem:

```
1 table {
2   width: 100%;
3   /* Overflows on mobile */
4 }
```

Solution:

```
1 .table-container {
2   overflow-x: auto;
3 }
4
5 @media (max-width: 768px) {
6   /* Convert to card layout or horizontal scroll */
7 }
```

7.4 Mistake 4: Insufficient Color Contrast

Problem:

```
1 table {
2   background: #fff;
3   color: #ccc; /* Insufficient contrast */
4 }
```

Correct:

```
1 table {  
2     background: #fff;  
3     color: #333; /* Good contrast */  
4 }
```

7.5 Mistake 5: Ignoring Accessibility

Bad:

```
1 <table>  
2     <tr><td>Header</td></tr>  
3 </table>
```

Good:

```
1 <table>  
2     <caption>Table description</caption>  
3     <thead>  
4         <tr><th scope="col">Header</th></tr>  
5     </thead>  
6 </table>
```

8 Quick Reference

8.1 Table Styling Patterns

```
1 /* Clean table */  
2 table {  
3     width: 100%;  
4     border-collapse: collapse;  
5 }  
6  
7 th, td {  
8     padding: 0.75rem;  
9     text-align: left;  
10    border-bottom: 1px solid #ddd;  
11 }  
12  
13 /* Striped rows */  
14 tbody tr:nth-child(even) {  
15     background: #f9f9f9;  
16 }  
17  
18 /* Hover effect */  
19 tbody tr:hover {  
20     background: #f0f0f0;  
21 }  
22  
23 /* Fixed header */  
24 thead {  
25     position: sticky;
```

```
26     top: 0;
27     background: white;
28     z-index: 10;
29 }
30
31 /* Responsive scroll */
32 .table-container {
33     overflow-x: auto;
34 }
```

8.2 Production Checklist

- Use semantic table structure (thead, tbody, tfoot)
- Use th elements for headers, not td
- Include table captions for descriptions
- Use scope attribute on headers
- Ensure sufficient color contrast
- Make tables responsive on mobile
- Test with screen readers
- Avoid using tables for layout
- Use border-collapse: collapse
- Optimize for performance with table-layout: fixed

9 Summary

CSS table properties enable proper presentation and styling of data tables. Key takeaways:

1. Use semantic HTML for table structure
2. Use border-collapse: collapse for clean tables
3. Implement responsive design for mobile
4. Ensure accessibility with proper headers and scope
5. Add visual hierarchy with color and spacing
6. Use sticky headers for long tables
7. Never use tables for layout
8. Test readability and contrast
9. Optimize with fixed table-layout when appropriate
10. Include alt text and descriptions

Master CSS table properties to create accessible, beautiful data presentations that work seamlessly across all devices while maintaining semantic HTML structure.